SECURITY NINJA GERMANY



Blue - Public

MICROSOFT SECURITY NINJA GERMANY · 20 OCTOBER

Shift-left is great, until your app owners haven't heard of it

Code to runtime container security with Microsoft Defender for Cloud

Nine gates, one digest

Four acts, four demos

Laptop to cluster door



Thank you to our Sponsors!

Host Partner

Microsoft

Gold

glueck kanja

water IT Security & Defense

robopack

PROACT

Silver

WORKSIMPLE EINFACH ARBEITEN

semperis

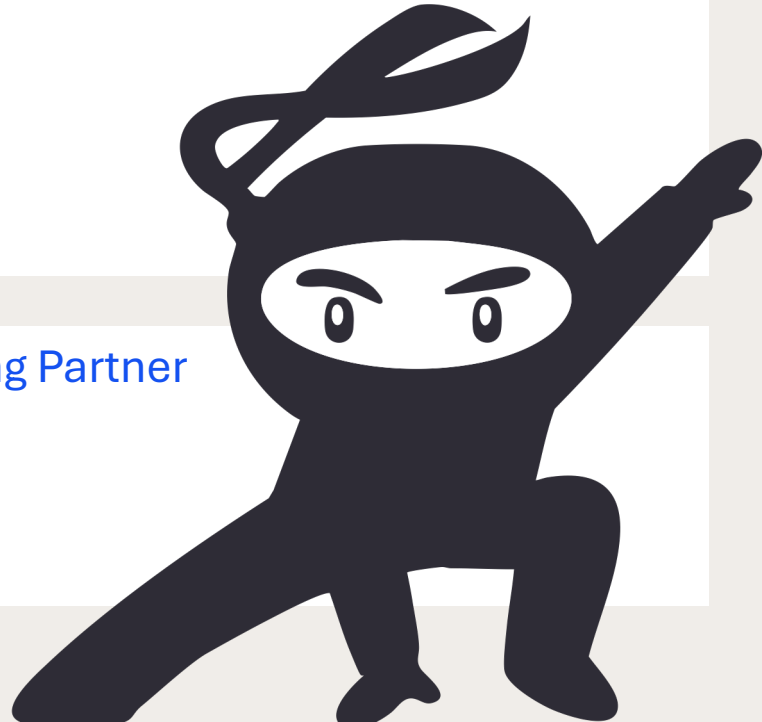
Bronze

ugurlabs

IC CONSULT

Snacks & Networking Partner

SCE



Blue - Public

SECURITY NINJA GERMANY



Pit Singert

Security Architect & Consultant · Water IT Security

15+ years in IT, almost 10 of them at Microsoft. XDR, SIEM, SOAR, agentic SOC, detection & hunting, and container security across the Microsoft stack.

LinkedIn
linkedin.com/in/pit-singert
Say hi, questions welcome

Blog
pisinger.github.io
PS, Here's What I Learned — gating, VAP, KQL

MS Release Radar
pisinger.tngx-voice.com/ms-release-radar
Defender and Azure release notes, weekly

GitHub
github.com/pisinger
This lab and more

Blue - Public

Why nobody talks about this

SECURITY NINJA GERMANY

Identity

Has an owner in the security team.

Endpoint

Has an owner in the security team.

Kubernetes

Platform owns the cluster. App teams own the images. Security owns neither — and gets the incident.

Admission control is the one place security can own a decision **without owning the pipeline or the cluster**.

Blue - Public

SECURITY NINJA GERMANY

Assume breach. Assume every gate fails eventually, because reviewers have bad days and pipelines have blind spots.

> Layered controls make sure one failure stays one failure.



The lab

One GitHub repo, one resource group, one Defender.

GitHub

Repo + push protection, secret & code scanning

Actions: build, Defender CLI, SARIF

Flux source: app/k8s

`OIDC → no secrets`

→

`Azure · one resource group`

Container Registry
Images + signed findings

AKS
Flux, sensor, gate, CEL

Container Apps
Control panel

Storage
Synthetic sensitive data

4 managed identities
CI, panel, app, gating

Optional
Foundry model, Log Analytics

AKS namespaces: x-mdc-blocked gated · x-mdc-allowed out of scope · demo-ops tooling

→

Defender

CSPM + DevOps connector

Containers: VA, gating, sensor

Sensitive data discovery

XDR: incidents, hunting

Bicep + one deploy workflow. Parked at night, started in the morning.

https://pisinger/ninja-defender-container-gating

The control panel

aca-webapp on Azure Container Apps. Today's cockpit.

SECURITY NINJA GERMANY

Cluster overview

Pods, images, Flux status, Defender alerts

kubectl console

Constrained, demo namespaces only, audited

Alert simulation

33 benign scenarios: drift, miner, network scan…

Admission check

Dry-run a deployment against the gate

AI prompt window

Optional Foundry model, guarded by Defender for AI

No secrets

Managed identity to AKS, Entra sign-in

It is also a target: internet-exposed, its own image scanned, its own identity raising alerts.

The lab is public — deploy it, sign in, and break things yourself.

Blue - Public

Kubernetes in 60 seconds

Eight words you will hear today.

SECURITY NINJA GERMANY

Image

The packaged app — like an installer. Its **digest** is the file hash.

Registry

Where images live — a software library. Here: Azure Container Registry.

Pod

One running copy of the app. Own IP, short-lived.

Deployment

“Keep three copies running.” Kill a pod and it comes back.

Namespace

A folder for one team's apps — like an OU or resource group.

Cluster / node

The fleet of servers (nodes) that runs all the pods.

Admission

The bouncer at the cluster's front door. Every create is checked first.

GitOps (Flux)

The cluster pulls its desired state from git — no one runs a deploy.

Read along: image in a registry → admission at the door → pods in a namespace.

Blue - Public

You already run this playbook

New nouns, familiar controls.

SECURITY NINJA GERMANY

Admission control	≈	Conditional Access, for workloads instead of users
Namespace tier	≈	Trust level / device compliance state
Image digest	≈	File hash — what the verdict is attached to
Binary drift	≈	Tamper detection on a device
Isolate pod	≈	Device isolation in Defender for Endpoint

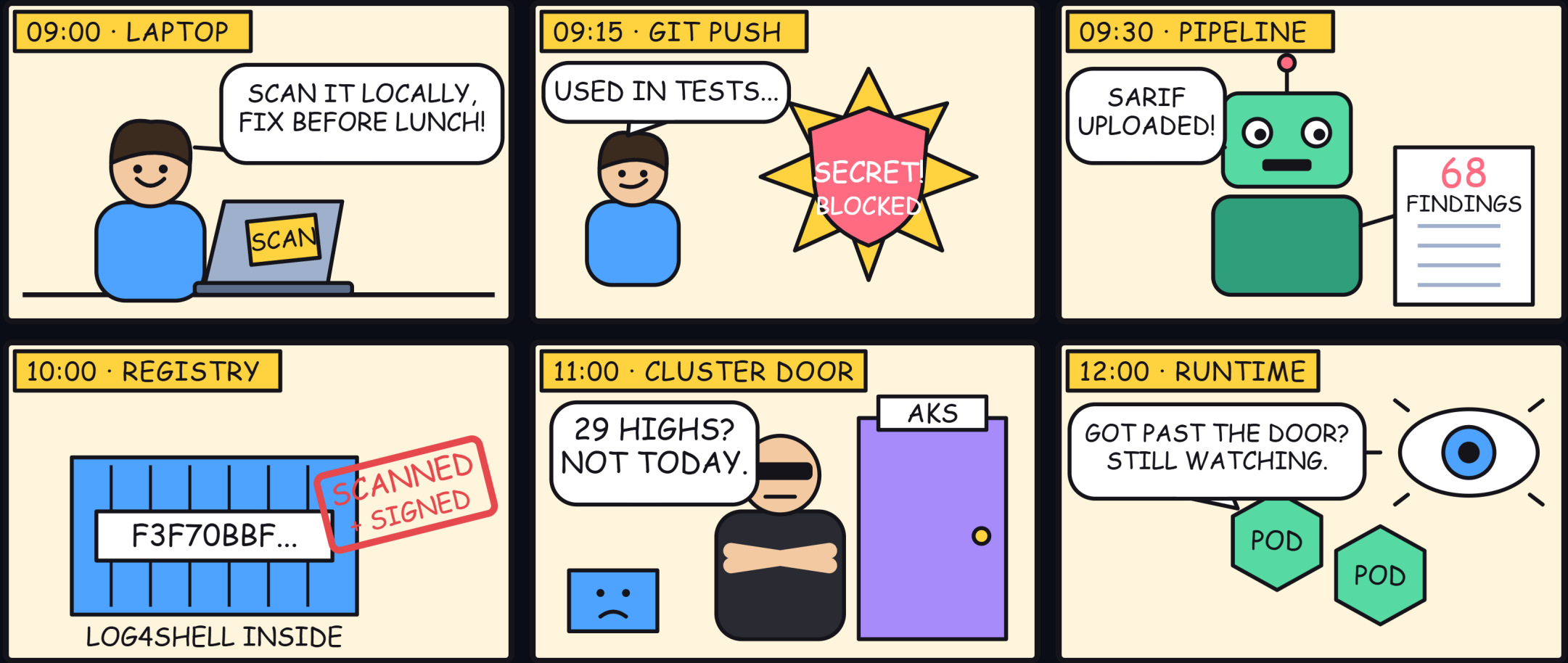
Same instincts you use for users and laptops — pointed at workloads.

Blue - Public

From dev to runtime

One image, one morning.

SECURITY NINJA GERMANY



Blue - Public

CI/CD and GitHub in 60 seconds

Eight more words.

SECURITY NINJA GERMANY

Repository
The project folder in GitHub, with every change ever made.

Commit & push
Save a change on the laptop → send it to GitHub.

Pull request
“Please review and merge my change.” Checks run here.

CI/CD
Build and test every change (CI), ship it automatically (CD).

Workflow
GitHub Actions recipe: build, scan, push the image — on every push.

OIDC
The pipeline logs in to Azure without a stored password.

Push protection
GitHub refuses a push that contains a secret.

SARIF
The standard scan report. GitHub shows it as code scanning alerts.

Read along: commit & push → pull request → workflow builds, scans and ships the image.

Blue - Public

SECURITY NINJA GERMANY

HANDS UP

Who has a pre-commit hook that *everyone* actually runs?

Keep your hand up if it cannot be skipped with --no-verify.



Blue - Public

The far left

Cheapest fix. Weakest control.

`gate 1 · laptop`

Local scan

Same Defender CLI as CI. No wasted build, no red Action run

`git commit`

Hook

Optional, gone with `--no-verify`

`git push`

Push protection

Server side. Lab: AWS key pair, 3 storage keys

`one click`

Bypass

Anyone with write access, reason of their choice

Bypass reason	What security is left with
Used in tests · False positive	Alert resolved in the same second
Fix later	Alert stays open, flows on to Defender

Make the left fast enough that nobody wants the bypass.

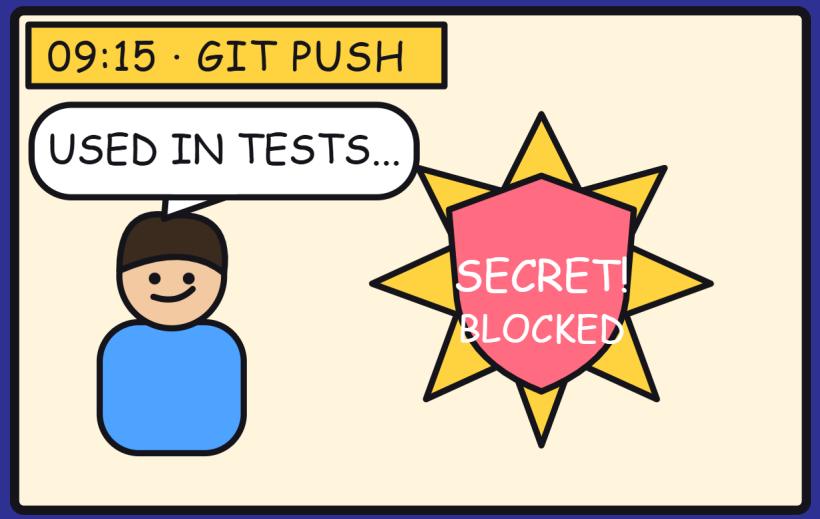
Delegated bypass turns the door into an approval — organisation repos with Secret Protection only.

↳ If push protection is bypassed → Gitleaks + Defender credscan in CI, alert in Defender

SECURITY NINJA GERMANY

DEMO 1

A secret, a push, a bypass



- The hook catches the secret — --no-verify skips it
- GitHub push protection blocks the push
- Bypass with a reason — then look at what security got

Blue - Public

Scan the code, two ways

In the pipeline if you wire it in. From the connector if you don't.

Defender CLI · CI and laptops

scan image	Image CVEs, same engine as the registry
scan sbom	SBOM + malicious packages
scan model	AI model files

Break the build | SARIF → GitHub | Upload → Defender

Agentless scanning · GitHub connector

code	Python, JS/TS (Bandit, ESLint)
deps	npm, pip, Maven, NuGet… (Trivy)
IaC	Bicep, Terraform, K8s, Dockerfile
SBOM	Queryable in Cloud Security Explorer

No pipeline edit | Daily, main branch | Cannot block

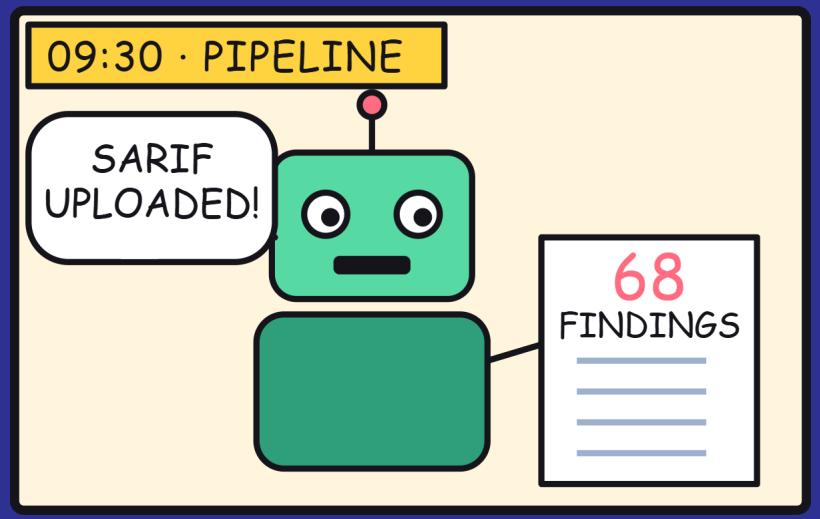
A scan that only prints to a build log is a scan nobody else can see.

↳ If the CI scan is skipped → the connector still scans the repo, the registry scans every push

DEMO 2

Scan, gate, report

- The CLI gate fails on Log4Shell — findings still reach code scanning
- The SBOM step: the line that talks to Defender
- Defender DevOps: the repo scanned without any pipeline change



Blue - Public

SECURITY NINJA GERMANY

HANDS UP

This image has a critical CVE. Who lets it into production?

Now keep your hand up if it is an internal batch job with no ingress.

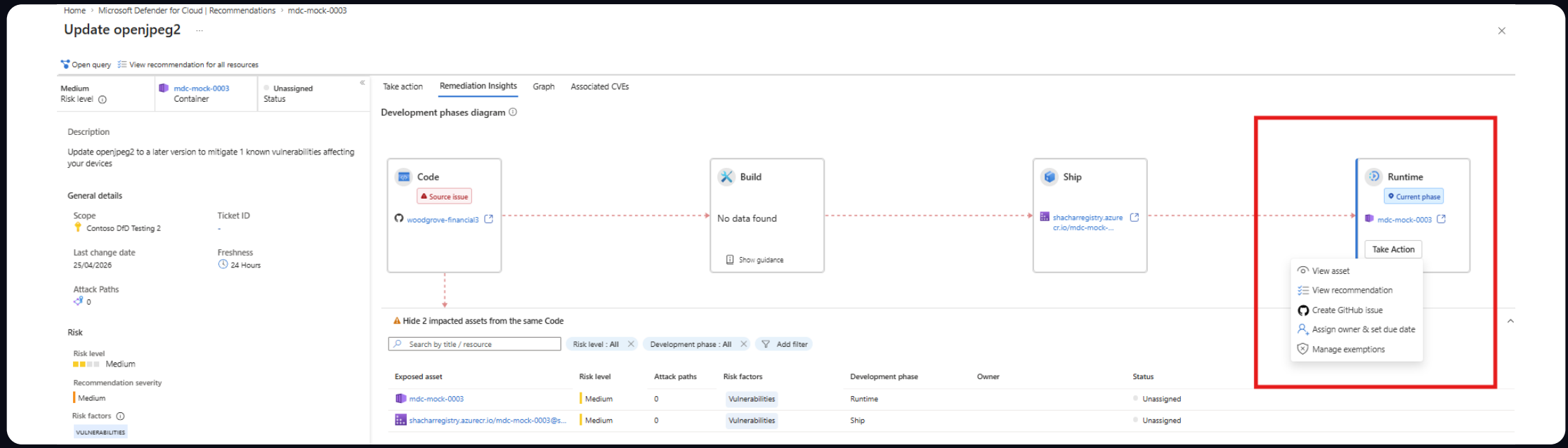


Blue - Public

Who owns this pod?

Code → Build → Ship → Runtime, and back to the team.

Shared cluster, many teams, one vulnerable container — and nobody knows whose it is.



Pod → image → repo, one click

Known to engineering? Related GitHub alerts

GitHub issue in the repo of origin

Campaigns by runtime risk

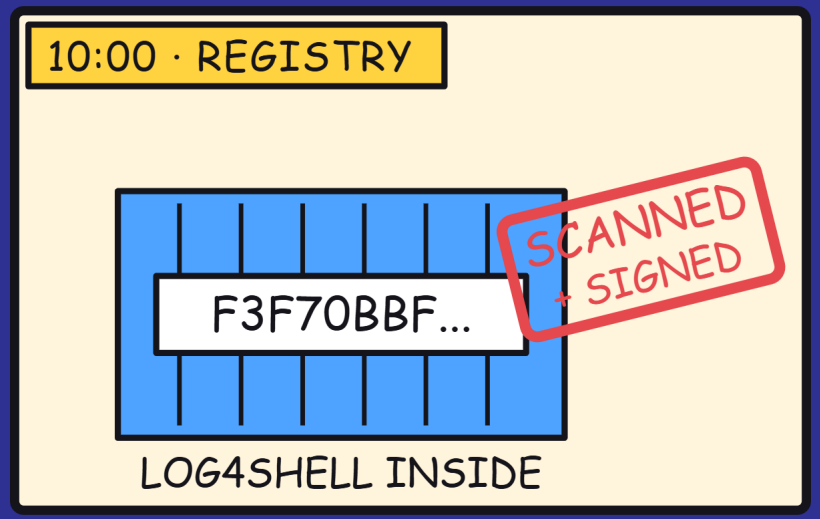
Image: Microsoft Learn · Deploy GitHub Advanced Security integration

SECURITY NINJA GERMANY

DEMO 3

Follow the digest

- Update log4j-core: one recommendation, severity versus risk
- Cloud Security Explorer: pushed by which pipeline, running where
- Repository findings: the bypassed secrets, the uploaded SARIF



Blue - Public

Two admission controls

Same namespace, different question.

	Gated deployment	Misconfiguration enforcement
Asks	Is this image too vulnerable?	Is this pod spec unsafe?
Checks	CVE severity, unscanned images, exemptions	Privileged, root, host namespaces, capabilities
Runs as	Defender webhook, reads signed findings	ValidatingAdmissionPolicy, CEL in the API server
Needs	Sensor, gating, registry access, findings · AKS 1.31+	Kubernetes API access · K8s 1.30+
Watch out	Fail-open while the webhook boots	No webhook to wait for

Both: GA, audit by default, deny per scope. The CEL policy stops the privileged pod even when its image is clean.

↳ If the webhook is down → CEL still enforces, the sensor still watches

Blue - Public

Block, or not?

Vulnerable is a fact about the image. **Exploitable is a fact about the path.**

	Edge ingress, web server	Internal backend, batch	Sandbox dev, tooling
Critical	Deny	Deny	Audit
High	Deny	Audit + ticket	Audit
Medium	Audit + ticket	Audit	Allow

SHIFT ONE STEP

- ↑ Known exploit (KEV)
- ↑ Identity reaches Azure
- ↓ WAF / App Gateway covers the vector
- ↓ No fix yet: dated exemption

The gate knows severity and scope, not exposure. **So exposure becomes the namespace.**

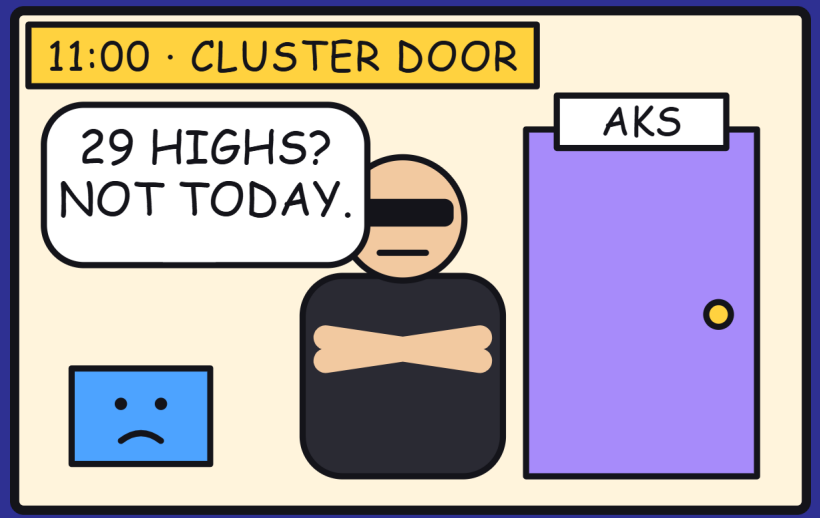
↳ If an exemption was wrong → the runtime sensor still watches the pod

DEMO 4

The door

```
denied: Image contains 29 high or higher CVEs, which is more
than the allowed count of: 0
```

- Flux syncs the hero digest into the gated namespace — denied
- Clean image, privileged pod — refused by the CEL policy
- Same hero digest, namespace outside the rule — admitted



Blue - Public

Note: The gate has a boot time

This morning, in the lab.

08:39	08:43	08:47	08:52
Cluster started	Nodes ready	Flux recreates the denied deployment — admitted	Defender admission controller up

`failurePolicy: Ignore`

No answer means yes. And admission judges create and update — never what already runs.

- Hold GitOps until the gate is ready
- Sweep gated namespaces after start or upgrade
- Alarm on the gate **not** enforcing

↳ Gate 7 missed it → gate 8, runtime vulnerability assessment, listed the pod

Blue - Public

One gate, three answers

Two policies per tier: severity, and evidence.

SECURITY NINJA GERMANY

edge

Internet-facing, user data

DENY critical and high

Unscanned: DENY

internal

Cluster-internal, no ingress

DENY critical · AUDIT high

Unscanned: AUDIT

sandbox

Dev, experiments, platform tooling

AUDIT everything

Unscanned: ALLOW

A missing findings artifact is **not** a clean image.

Block deployments with missing artifacts, and a scanning delay stops being a hole.

```
≈ namespace tier = device compliance state
```

Exposure decides, not the CVSS number. Same digest, different namespace, different verdict.

Blue - Public

Watch the gates

Advanced Hunting and Resource Graph. Copy, paste, adapt.

SECURITY NINJA GERMANY

Every image gate decision

```
CloudPolicyEnforcementEvents
| where Timestamp > ago(1d)
| summarize Events = count() by ActionType,
    KubernetesNamespace, ResourceName, Reason
```

Misconfiguration denials (CEL)

```
CloudAuditEvents
| where OperationName == "create"
| extend R = RawEventData.ResponseStatus
| where toint(R.code) == 403
| where tostring(RawEventData.Annotations)
    contains "validation_failure"
```

Who hit the admission path

```
CloudAuditEvents
| where OperationName == "create"
| extend Msg = tostring(
    RawEventData.ResponseStatus.message)
| where Msg has "admission"
| project Timestamp, UserAgent, IPAddress, Msg
```

CVE enrichment: risk vs severity (ARG)

```
arg().securityresources
| where type =~ "microsoft.security/assessments"
| extend D = properties.additionalData
| where isnotempty(D.CvesDetails)
| project Pkg = D.SoftwareName, CVEs = D.CvesDetails,
    Risk = properties.risk.level
```

Full versions: pisinger.github.io · gated deployment (Aug 7) · misconfiguration with VAP (Aug 4)

Blue - Public

Yes, it looks complex. It is also a team sport.

SECURITY NINJA GERMANY

Cluster owner

The cluster and the policy. Keeps the gate armed — after every start.

Developers

Scan locally, fix at the source, once. Everything downstream inherits it.

Cloud security

Coverage that needs nobody's permission. Owns the exception, with an expiry.

SOC

Acts at 12:30 with context it did not have to ask for.

Pipelines, registries, clusters, PaaS, identities — nobody holds that in their head.

Defender consolidates it: one finding on one digest, read by the gate, the graph and the incident.

Never trust a single gate. Every gate needs a backup. **That is defence in depth.**

Blue - Public

A denial is a path, not a wall

Fix available

Bump it. Minutes, no exemption, no meeting.

No fix yet

Time-boxed exemption, recorded with an owner and a date.

Incident hotfix

Break-glass under 15 minutes, 24/7.

Wrong placement

Move the workload. The gate was right, the map was wrong.

```
≈ exemption = a policy exclusion with an expiry date
```

Every exemption is time-bound. **Expiry is the review** — an accepted risk never quietly becomes permanent.

Blue - Public

Nine gates, one digest

Each one bypassed. The next one catches it.

SECURITY NINJA GERMANY

1 Local scan + hooks
Saves builds; skipped with --no-verify

2 Push protection
One click, reason of choice

3 Repo, centrally
Nothing to bypass — no pipeline edit

4 Pipeline scan
skip_scan, force_push, no pipeline

5 Registry
Scanned anyway, findings signed on the digest

6 Security graph
Reads the estate, maps it to owners

7 Admission
Only via the rule — or before the gate is up

8 Runtime sensor
What got through is still watched

9 SOC
Context, Response Actions

■ Act 1 ■ Act 2 ■ Act 3 ■ Act 4

No single gate is the control. **The stack is.**

Blue - Public

Nine gates, four acts

Laptop to cluster door, one digest.

ACT 1

Developer

1 Local scan + hooks

2 Push protection

ACT 2

Repo + pipeline

3 Agentless repo scanning

4 Defender CLI in CI

ACT 3

Defender for Cloud

5 Registry scan, signed findings

6 Graph: code to runtime

ACT 4

Runtime

7 Admission: CVEs + misconfig

8 Runtime sensor

9 Response Actions

THE HERO DIGEST

`aks-webapp@sha256:f3f70bbf23cb…`

Log4Shell inside. Follow it through every act.

Every gate gets bypassed. **The stack is the control.**

Gate 9, the SOC: backup slides

Blue - Public

12:00 runtime → 12:30 the SOC

Hands up: **terminate** the pod, or **isolate** it?

WHY THIS IS HARDER THAN A SERVER

The pod comes back. GitOps puts it back. The identity reaches past the cluster.

- Shared nodes, many owners — isolating one thing touches someone else's workload
- Credential Access by a service principal: pod → workload identity → Azure → PaaS

SECURITY NINJA GERMANY

The Tools

Blue - Public

Defender CLI

One binary. Any pipeline, any laptop.

Command	Covers
scan image	Image CVEs — same MDVM engine as the registry
scan sbom	SBOM + malicious packages, not CVEs
scan model	AI model files (pickle, ONNX, ...)

Gate
--defender-break exit code

SARIF → GitHub
Lab: 68 findings, 27 high

Upload → Defender
Pipeline → digest in the graph

A scan that only prints to a build log is a scan nobody else can see.

Upload with if: always() — a failing gate is exactly when there is something to report.

↳ If the CI scan is skipped → agentless repo scan, then the registry scans every push

Blue - Public

Agentless repo scanning

One GitHub connector. Zero pipeline edits.

SECURITY NINJA GERMANY

Code

Bandit, ESLint · Python, JS/TS

Dependencies

Trivy · npm, pip, Maven, NuGet, Go…

IaC

Checkov, Template Analyzer · Bicep, Terraform, K8s, Helm, Dockerfile

SBOM

Syft · queryable in Cloud Security Explorer

GitHub alerts, imported

Secret scanning, code scanning incl. your SARIF, Dependabot

DevOps posture

Repo and pipeline settings, every 8 hours

Code to runtime

Repo → pipeline → image → workload

`default branch · daily scan · repos < 1 GB · cannot break a build · preview`

Coverage for every repository — including the ones whose pipeline nobody touched.

↳ It cannot block → that is the job of the registry scan and the admission gate

Blue - Public

What Defender actually reads

Four sources, one graph.

SECURITY NINJA GERMANY

GitHub connector

Secrets, code scanning, dependencies, IaC

Defender CLI

Which pipeline pushed which digest

Registry scan

Every pushed image, findings signed onto the digest

Runtime Protection

AKS pods, Container Apps, storage data

↓

Security graph → recommendations, attack paths, Cloud Security Explorer

One connector, zero pipeline edits. The picture is only as complete as what each source sends.

Blue - Public

From the registry to the SOC

Two surfaces, one digest.

SECURITY NINJA GERMANY

PREVENT

Defender for Containers

Registry vulnerability assessment

Scan on push, daily re-scan, OS and language packages

Signed findings artifact

Attached to the digest — the evidence the gate reads

Gated deployment

Admission webhook audits or denies, per rule scope

Admission monitoring

Which rule, which digest, which exemption applied

Defender sensor

Binary drift, threat detection, malware at runtime

RESPOND

SecOps with Defender XDR

Terminate pod

Stops it now — and takes the volatile evidence with it

Isolate pod

Cuts the reach, the process stays for forensics

Restrict pod

The middle option a blast-radius decision needs

Advanced Hunting

CloudPolicyEnforcementEvents, CloudAuditEvents — queryable

One incident, both sides

Cloud posture and endpoint-grade response in one queue

```
≈ isolate pod = device isolation, aimed at a workload
```

Findings are signed onto the **digest** — so prevention and response argue about the same object.

See you next Year!

